# SIMATS ENGINEERING

## ASSESSMENT TOOL 1: Knowledge Representation Design Assessment

| | | | |
|---|---|---|---|
| **COURSE NAME:** | **ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS** | **COURSE CODE:** | **MLA0101** |

---

**COURSE OUTCOME COVERED**

**CO3:** *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.*(BTL 3)

Assessment Tool Weightage: 40 %

---

**Total Marks: 100** **Time: 60 Mins**

**No. of Questions: 5**

Annexure A

## Knowledge Representation Design Assessment

---

***Code of Conduct:***

*I Shaik Mubarak(192425194)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** 

| Criterion | Weightage | Marks Obtained |
|---|---|---|
| 1. Problem Analysis and Domain Understanding | 10% | |
| 2. Knowledge Engineering and Knowledge Base Design | 15% | |
| 3. Propositional Logic Representation | 10% | |
| 4. First-Order Logic Representation | 15% | |
| 5. Unification and Resolution | 15% | |
| 6. Forward and Backward Chaining | 15% | |
| 7. Inference and Reasoning Accuracy | 10% | |
| 8. System Architecture / Representation Diagram | 5% | |
| 9. Prototype Implementation and Results | 5% | |
| Total | 100% | |

Signature of the Faculty Total Marks Awarded

## ANSWER ANY ONE QUESTION

## 1. Smart Healthcare Diagnosis System

A hospital wants to develop an AI-based system that can identify possible illnesses from patient symptoms. The system contains facts such as:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

**Task:**

1. Represent the given knowledge using **Propositional Logic**.
2. Convert the knowledge into **First-Order Logic (FOL)**.
3. Construct a knowledge base containing facts and rules.
4. Demonstrate **unification** for at least two predicates.
5. Use **forward chaining** to determine Ravi's condition.
6. Use **backward chaining** to prove whether Ravi needs rest.
7. Demonstrate **resolution** for one selected query.
8. Explain the final inference produced by the system.

## 2. Intelligent Loan Approval System

A bank wants to develop an AI system to determine whether a customer is eligible for a loan. The knowledge base contains information about income, credit score, employment, existing loans, and repayment history.

For example:

- Arun has a stable job.
- Arun has a high credit score.
- Arun has sufficient income.
- Customers with stable employment and sufficient income are eligible for preliminary approval.
- Customers with high credit scores and good repayment history are considered low-risk.
- Low-risk customers with preliminary approval can be recommended for loan approval.

**Task:**

1. Identify the **entities, facts, predicates, and rules** involved.
2. Represent the knowledge using **First-Order Logic**.
3. Build a knowledge base with at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.

5. Apply **forward chaining** to determine whether Arun can be recommended for approval.
6. Apply **backward chaining** for the query `LoanApproved(Arun)`.
7. Use **resolution** to prove or disprove one query.
8. Discuss how the knowledge-engineering process helps maintain the loan rules.

## 3. Intelligent Vehicle Fault Diagnosis

An automobile service center wants to develop an expert system that identifies possible vehicle faults based on symptoms.

The system knows that:

- A vehicle has a dead battery if it does not start and the headlights are dim.
- A vehicle may have a fuel problem if the engine cranks but does not start.
- A vehicle may have an overheating problem if the temperature is high and coolant is low.
- A vehicle should be sent for immediate service if overheating is detected.

For a vehicle named **Car101**, the system receives:

- Car101 does not start.
- Car101 has dim headlights.
- Car101 has a high temperature.

**Task:**

1. Design the **knowledge representation** for the vehicle diagnosis system.
2. Represent at least five rules using **FOL**.
3. Represent selected statements using **Propositional Logic**.
4. Demonstrate **unification** using suitable predicates.
5. Apply **forward chaining** to identify possible faults.
6. Apply **backward chaining** for the query `NeedsService(Car101)`.
7. Demonstrate **resolution** for one diagnosis.
8. Explain how inference is used to arrive at the final diagnosis.

## 4. Intelligent Student Academic Advising System

A university wants an AI-based academic advising system that recommends actions to students based on their academic performance.

The system contains rules such as:

- Students with attendance below 75% require attendance counseling.
- Students who fail two or more courses require academic counseling.
- Students with high attendance and good academic performance can be recommended for advanced courses.
- Students who have completed prerequisite courses can register for the corresponding advanced course.

Consider a student **Priya** who has:

- Attendance of 68%.
- Two failed courses.
- Completed the prerequisite for an advanced AI course.

**Task:**

1. Identify the **knowledge engineering requirements** for the system.
2. Represent the academic rules using **FOL**.
3. Create a knowledge base containing facts and rules.
4. Demonstrate **unification** using student and course predicates.
5. Use **forward chaining** to identify recommendations for Priya.
6. Use **backward chaining** to determine whether Priya requires academic counseling.
7. Apply **resolution** to prove one selected recommendation.
8. Explain the advantages and limitations of using a rule-based inference system for academic advising.

## 5. Cybersecurity Threat Detection System

An organization wants to develop an AI knowledge-based system to identify suspicious activities in its network.

The knowledge base contains rules such as:

- Multiple failed login attempts from the same user may indicate a brute-force attack.
- A login from an unusual location combined with multiple failed attempts indicates suspicious activity.
- Suspicious activity followed by unauthorized file access indicates a possible security breach.
- A possible security breach should generate a high-priority alert.

For user **User101**, the system observes:

- Five failed login attempts.
- Login from an unusual location.
- Unauthorized access to a confidential file.

**Task:**

1. Design a **knowledge representation model** for the cybersecurity domain.
2. Represent the knowledge using **Propositional Logic and FOL**.
3. Construct at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.
5. Apply **forward chaining** to determine whether a security alert should be generated.
6. Apply **backward chaining** for the query `HighPriorityAlert(User101)`.
7. Use **resolution** to prove the existence of a possible security breach.
8. Compare **forward chaining and backward chaining** for cybersecurity reasoning and justify which is more suitable.

## Evaluation Rubrics:

| Criterion | Weightage (%) | Excellent | Good | Average | Poor |
|---|---|---|---|---|---|
| Problem Analysis and Understanding | 15% | 13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference. | 10–12% – Understands the problem with minor omissions. | 7–9% – Shows partial understanding with some important elements missing. | 0–6% – Poor or incorrect understanding of the problem. |
| Knowledge Engineering & Knowledge Base Design | 15% | 13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules. | 10–12% – Knowledge base is well designed with minor inconsistencies or omissions. | 7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear. | 0–6% – Knowledge base is incomplete, inconsistent, or inappropriate. |
| Propositional & First-Order Logic Representation | 20% | 17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy. | 13–16% – Logic representation is mostly correct with minor errors. | 9–12% – Provides basic representations but has several logical or syntactic errors. | 0–8% – Logic representation is incorrect, incomplete, or missing. |
| Unification & Resolution | 15% | 13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions. | 10–12% – Applies unification and resolution correctly with minor errors. | 7–9% – Demonstrates partial understanding but misses important steps or contains errors. | 0–6% – Unable to correctly apply unification or resolution. |

| | | | | | |
|---|---|---|---|---|---|
| Forward Chaining & Backward Chaining | 15% | 13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection. | 10–12% – Correctly applies both methods with minor errors or omissions. | 7–9% – Demonstrates partial understanding of one or both chaining methods. | 0–6% – Incorrectly applies or fails to demonstrate chaining techniques. |
| Inference & Reasoning Accuracy | 10% | 9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference. | 7–8% – Conclusions are mostly correct with minor reasoning gaps. | 5–6% – Some correct inferences but reasoning is incomplete or unclear. | 0–4% – Conclusions are incorrect, unsupported, or missing. |
| Knowledge Representation Diagram / System Architecture | 5% | 5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output. | 4% – Diagram is mostly correct with minor omissions. | 3% – Diagram is partially complete or lacks clarity. | 0–2% – Diagram is missing or incorrect. |
| Originality, Critical Thinking & Presentation | 5% | 5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation. | 4% – Demonstrates good reasoning and clear presentation. | 3% – Shows limited critical thinking and basic presentation. | 0–2% – Shows little originality, weak reasoning, or poor presentation. |

# 5.CYBERSECURITY THREAT DETECTION SYSTEM USING KNOWLEDGE REPRESENTATION AND INFERENCE

## 1. Introduction

Cybersecurity threats such as brute-force attacks, suspicious logins, unauthorized file access, and security breaches are major concerns for modern organizations. A knowledge-based artificial intelligence system can help detect such threats by representing cybersecurity knowledge as facts and logical rules.

This case study develops an intelligent cybersecurity threat detection system for identifying suspicious activities associated with a user. The system uses Propositional Logic, First-Order Logic (FOL), Knowledge Base construction, Unification, Substitution, Forward Chaining, Backward Chaining, and Resolution to determine whether a high-priority security alert should be generated.

For this case study, the system observes the activities of User101:

- User101 has five failed login attempts.
- User101 logged in from an unusual location.
- User101 accessed a confidential file without authorization.

The objective is to determine whether these activities indicate a possible security breach and whether a high-priority alert should be generated.

## 2. Problem Analysis and Understanding

### Problem Statement

An organization wants to develop an AI-based knowledge system that can automatically analyze network activities and identify suspicious behavior.

The system should:

1. Detect multiple failed login attempts.
2. Identify unusual login locations.
3. Detect unauthorized file access.
4. Infer suspicious activity.
5. Identify possible security breaches.
6. Generate a high-priority alert when a security breach is detected.

### Main Objective

The main objective is to use logical reasoning and inference techniques to transform observed cybersecurity events into meaningful security conclusions.

### Input

The system receives cybersecurity events such as:

- Failed login attempts.
- Unusual login location.
- Unauthorized file access.
- Confidential file access.

### Output

The expected output is:

**HighPriorityAlert(User101)**

This means that the system should generate a high-priority security alert for User101.

## 3. Knowledge Engineering Requirements

Knowledge engineering is the process of collecting, representing, organizing, and maintaining knowledge so that an AI system can reason about a particular domain.

For this cybersecurity system, the major requirements are:

### 3.1 Entities

The important entities are:

- Users
- Login attempts
- Locations

- Files
- Security events
- Security alerts

Example entity:

**User101**

### 3.2 Predicates

Predicates describe properties or relationships.

Examples:

- FailedLogin(User101)
- UnusualLocation(User101)
- UnauthorizedAccess(User101)
- ConfidentialFile(File101)
- SuspiciousActivity(User101)
- SecurityBreach(User101)
- HighPriorityAlert(User101)

### 3.3 Facts

Facts represent information directly observed by the system.

Example: FiveFailedLogins(User101)

### 3.4 Rules

Rules represent cybersecurity knowledge.

Example: FiveFailedLogins(x) → BruteForceAttack(x)

### 3.5 Inference Engine

The inference engine applies rules to known facts to derive new information.

The system uses:

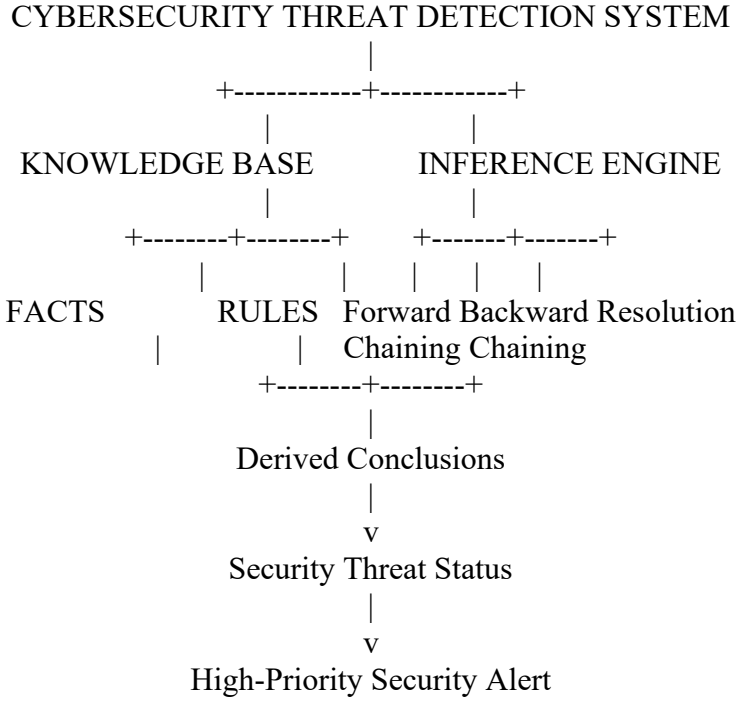
- Forward Chaining
- Backward Chaining
- Resolution

### 3.6 Knowledge Base

The knowledge base stores all facts and rules required for reasoning.

## 4. Knowledge Representation Model

The cybersecurity system can be represented using the following structure:



**System Architecture Explanation**
The Knowledge Base contains cybersecurity facts and rules.
The Inference Engine processes these facts using logical reasoning techniques.
The system first receives observed events such as failed logins and unauthorized access. The inference engine applies the appropriate rules and derives new conclusions.
Finally, if the system derives:
SecurityBreach(User101)
then it concludes:
HighPriorityAlert(User101)

**5. Propositional Logic Representation**
In Propositional Logic, each statement is represented using a proposition.
Let:

- **P** = User101 has five failed login attempts.
- **Q** = User101 logged in from an unusual location.
- **R** = User101 performed unauthorized access to a confidential file.
- **S** = User101 is involved in a brute-force attack.
- **T** = User101 has suspicious activity.
- **U** = User101 has a possible security breach.
- **V** = A high-priority alert should be generated for User101.

**Propositional Rules**

1. Five failed login attempts indicate a possible brute-force attack:

**P → S**

2. A brute-force attack indicates suspicious activity:

**S → T**

3. Unusual location combined with multiple failed login attempts indicates suspicious activity:

**P ∧ Q → T**

4. Suspicious activity combined with unauthorized file access indicates a possible security breach:

**T ∧ R → U**

5. A possible security breach generates a high-priority alert:

**U → V**

**Initial Facts**
**P = True**
**Q = True**
**R = True**
Therefore:
P → S
S → T
T ∧ R → U
U → V
Hence:
**V = True**
Therefore, a high-priority security alert should be generated.

**6. First-Order Logic Representation**
First-Order Logic provides a more expressive representation because it allows variables, predicates, constants, and quantifiers.
**Constants**

- **User101** – represents the user.
- **File101** – represents a confidential file.

**Predicates**

| Predicate | Meaning |
|---|---|
| FiveFailedLogins(x) | x has five or more failed login attempts |
| UnusualLocation(x) | x logged in from an unusual location |
| BruteForceAttack(x) | x may be involved in a brute-force attack |
| SuspiciousActivity(x) | x shows suspicious activity |
| UnauthorizedAccess(x) | x performed unauthorized file access |
| ConfidentialFile(y) | y is a confidential file |
| SecurityBreach(x) | x may be involved in a security breach |
| HighPriorityAlert(x) | a high-priority alert should be generated for x |
| Login(x) | x performed a login |
| NormalLocation(x) | x logged in from a normal location |

**7. Knowledge Base**

The knowledge base consists of facts and rules.

**7.1 Facts**

The following facts are stored in the knowledge base:

1. FiveFailedLogins(User101)
2. UnusualLocation(User101)
3. UnauthorizedAccess(User101)
4. ConfidentialFile(File101)
5. Login(User101)
6. UserActive(User101)
7. SecurityMonitoringEnabled(User101)
8. FileAccessLogged(User101)
9. LoginAttemptRecorded(User101)
10. AccountExists(User101)
11. FileExists(File101)
12. ConfidentialAccessMonitored(File101)

Thus, the system contains more than the required 10 facts.

**8. Knowledge Base Rules**

The system contains the following rules:

**Rule 1 – Brute-Force Detection**

If a user has multiple failed login attempts, the user may be involved in a brute-force attack. **∀x FiveFailedLogins(x) → BruteForceAttack(x)**

**Rule 2 – Suspicious Activity from Brute Force**

If a user is involved in a brute-force attack, the activity is suspicious.

**∀x BruteForceAttack(x) → SuspiciousActivity(x)**

**Rule 3 – Unusual Login Detection**

If a user logs in from an unusual location, the login is suspicious.

**∀x UnusualLocation(x) → SuspiciousLogin(x)**

**Rule 4 – Combined Suspicious Activity**

If a user has five failed login attempts and logs in from an unusual location, suspicious activity is detected.

**∀x FiveFailedLogins(x) ∧ UnusualLocation(x) → SuspiciousActivity(x)**

**Rule 5 – Unauthorized Confidential File Access**

If a user performs unauthorized access and the accessed file is confidential, the activity is suspicious.

**∀x∀y UnauthorizedAccess(x) ∧ ConfidentialFile(y) → SuspiciousFileActivity(x)**

**Rule 6 – Security Breach Detection**

If suspicious activity occurs together with unauthorized access, a possible security breach is detected.

**∀x SuspiciousActivity(x) ∧ UnauthorizedAccess(x) → SecurityBreach(x)**

**Rule 7 – High-Priority Alert**

If a security breach is detected, a high-priority alert should be generated.

**∀x SecurityBreach(x) → HighPriorityAlert(x)**

**Rule 8 – Immediate Security Investigation**

If a high-priority alert is generated, the user's activity should be investigated.

**∀x HighPriorityAlert(x) → InvestigateUser(x)**

**9. Unification and Substitution**

Unification is the process of making two logical expressions identical by finding suitable substitutions for their variables.

**Example 1**

Consider:

**FiveFailedLogins(x)**

and

**FiveFailedLogins(User101)**

These predicates can be unified by substituting:

**x = User101**

Therefore:

**{x/User101}**

After substitution:

**FiveFailedLogins(User101)**

Hence, the two predicates successfully unify.

**Example 2**

Consider:

**UnauthorizedAccess(x)**

and

**UnauthorizedAccess(User101)**

The substitution is:

**{x/User101}**

After substitution:

**UnauthorizedAccess(User101)**

Therefore, unification succeeds.

**Most General Unifier**

The Most General Unifier (MGU) for both examples is:

**θ = {x/User101}**

This substitution allows the general rule to be applied to User101.

**10. Forward Chaining**

Forward chaining is a data-driven inference technique. It starts with known facts and repeatedly applies rules to derive new facts.

**Initial Facts**

F1: FiveFailedLogins(User101)
F2: UnusualLocation(User101)
F3: UnauthorizedAccess(User101)
F4: ConfidentialFile(File101)

**Step 1 – Detect Brute-Force Attack**

Rule 1:
FiveFailedLogins(x) → BruteForceAttack(x)
Since:
FiveFailedLogins(User101)
Therefore:
BruteForceAttack(User101)

**Step 2 – Detect Suspicious Activity**

Rule 2:

BruteForceAttack(x) → SuspiciousActivity(x)
Therefore:
SuspiciousActivity(User101)

**Step 3 – Confirm Suspicious Activity**

Rule 4:
FiveFailedLogins(x) ∧ UnusualLocation(x) → SuspiciousActivity(x)
Both conditions are true for User101.
Therefore:
SuspiciousActivity(User101)

**Step 4 – Detect Security Breach**

Rule 6:
SuspiciousActivity(x) ∧ UnauthorizedAccess(x) → SecurityBreach(x)
Since both conditions are true:
SecurityBreach(User101)

**Step 5 – Generate Alert**

Rule 7:
SecurityBreach(x) → HighPriorityAlert(x)
Therefore:
HighPriorityAlert(User101)

**Forward Chaining Result**

User101 → Brute-Force Attack → Suspicious Activity → Security Breach → High-Priority Alert

**11. Backward Chaining**

Backward chaining is a goal-driven inference technique. It starts with a desired conclusion and works backward to determine whether the required conditions are available.

**Query**

**HighPriorityAlert(User101)**

We need to determine whether this query is true.

**Step 1**

Find a rule whose conclusion is:
HighPriorityAlert(x)
Rule 7:
SecurityBreach(x) → HighPriorityAlert(x)
Therefore, we need to prove:
SecurityBreach(User101)

**Step 2**

Find a rule that concludes SecurityBreach(x).
Rule 6:
SuspiciousActivity(x) ∧ UnauthorizedAccess(x) → SecurityBreach(x)
Therefore, we need to prove:

1. SuspiciousActivity(User101)
2. UnauthorizedAccess(User101)

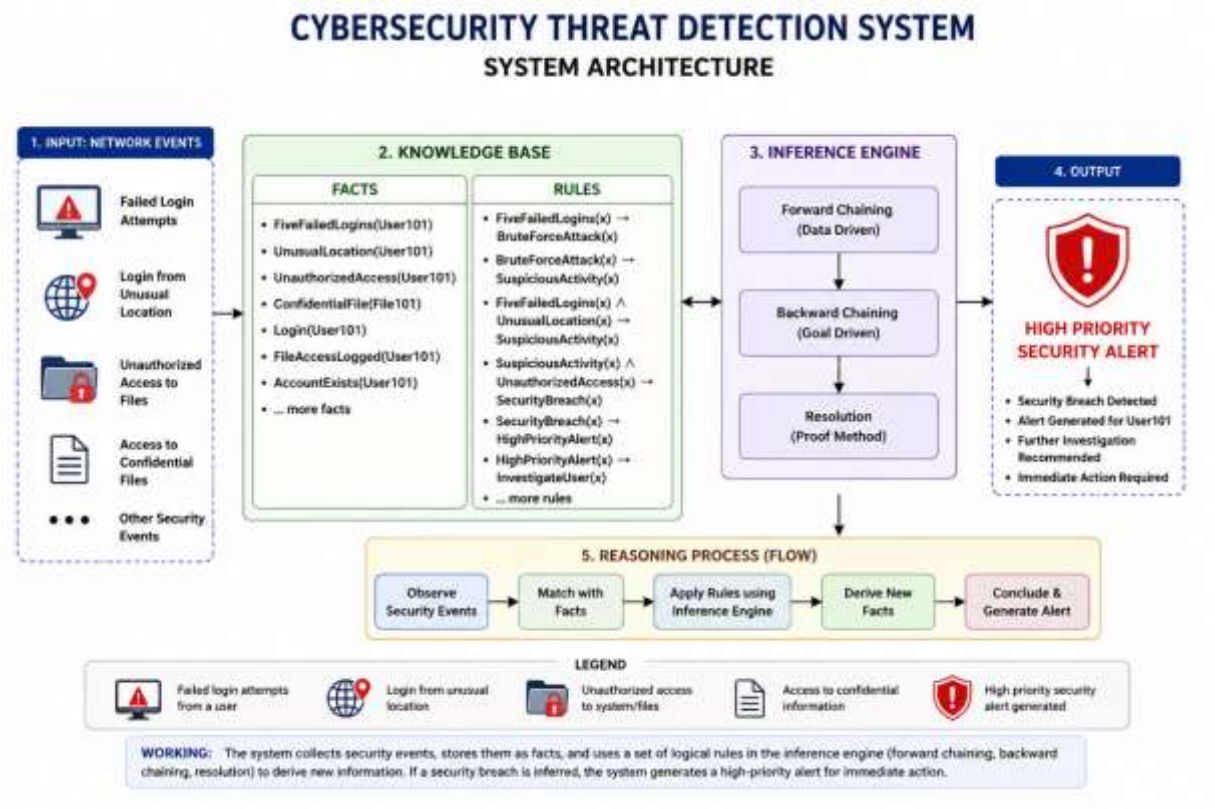
**Step 3**

Check the knowledge base.
UnauthorizedAccess(User101) is already a fact.

**Step 4**

To prove:
SuspiciousActivity(User101)
Use Rule 2:
BruteForceAttack(x) → SuspiciousActivity(x)
To prove BruteForceAttack(User101), use Rule 1:
FiveFailedLogins(x) → BruteForceAttack(x)
The fact:

FiveFailedLogins(User101)
is available.
Therefore:
BruteForceAttack(User101)
and hence:
SuspiciousActivity(User101)
Final Backward Chaining Result
Since both:
SuspiciousActivity(User101)
and
UnauthorizedAccess(User101)
are true:
SecurityBreach(User101) is true.
Therefore:
HighPriorityAlert(User101) is true.




**12. Resolution**

Resolution is an inference technique used to prove whether a query logically follows from a knowledge base.

To prove:

**SecurityBreach(User101)**

we use the rule:

SuspiciousActivity(x) ∧ UnauthorizedAccess(x) → SecurityBreach(x)

Convert the implication into a clause:

¬SuspiciousActivity(x) ∨ ¬UnauthorizedAccess(x) ∨ SecurityBreach(x)

For User101:

¬SuspiciousActivity(User101) ∨ ¬UnauthorizedAccess(User101) ∨ SecurityBreach(User101)

Known facts:

SuspiciousActivity(User101)

UnauthorizedAccess(User101)

**Resolution Step 1**

Resolve:

¬SuspiciousActivity(User101) ∨ ¬UnauthorizedAccess(User101) ∨ SecurityBreach(User101)
with:
SuspiciousActivity(User101)
This gives:
¬UnauthorizedAccess(User101) ∨ SecurityBreach(User101)

**Resolution Step 2**

Resolve with:
UnauthorizedAccess(User101)
This gives:
SecurityBreach(User101)
Therefore:
SecurityBreach(User101) is logically proven.

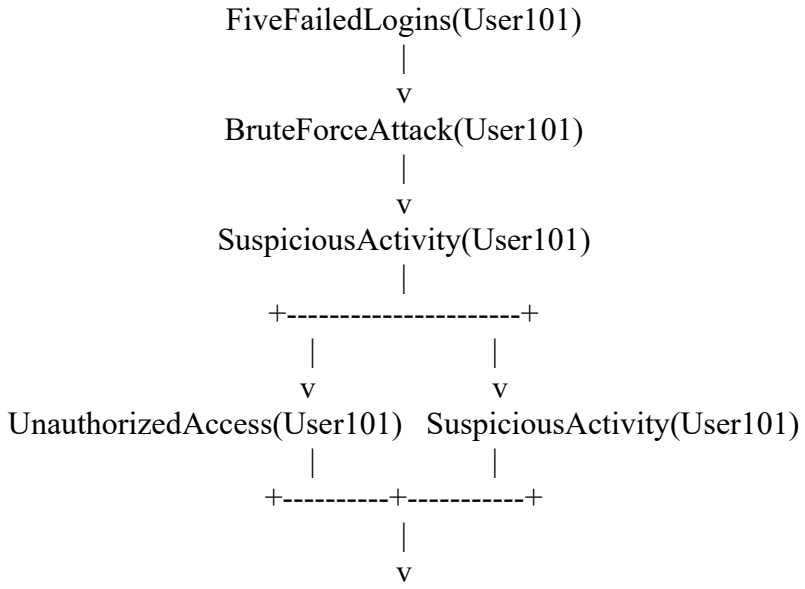
**13. Resolution by Refutation for High-Priority Alert**

To prove:
**HighPriorityAlert(User101)**
assume the opposite:
**¬HighPriorityAlert(User101)**
From Rule 7:
**SecurityBreach(x) → HighPriorityAlert(x)**
Clause form:
**¬SecurityBreach(x) ∨ HighPriorityAlert(x)**
For User101:
**¬SecurityBreach(User101) ∨ HighPriorityAlert(User101)**
We have already derived:
**SecurityBreach(User101)**
Resolving this with:
**¬SecurityBreach(User101) ∨ HighPriorityAlert(User101)**
gives:
**HighPriorityAlert(User101)**
This contradicts the assumed negation:
**¬HighPriorityAlert(User101)**
Therefore, the query is proven:
**HighPriorityAlert(User101)**

---

**14. Complete Inference Chain**

The complete reasoning process can be represented as:



SecurityBreach(User101)
|
v
HighPriorityAlert(User101)
|
v
InvestigateUser(User101)

---

**15. Forward Chaining vs Backward Chaining**

| Feature | Forward Chaining | Backward Chaining |
|---|---|---|
| Reasoning approach | Data-driven | Goal-driven |
| Starting point | Known facts | Query/goal |
| Direction | Facts → conclusion | Goal → required facts |
| Suitable for | Continuous monitoring | Specific investigation |
| Cybersecurity use | Real-time threat detection | Investigating a particular alert |
| Example | Detecting attacks from incoming events | Checking whether User101 needs an alert |

**Which is More Suitable?**

For cybersecurity threat detection, forward chaining is highly suitable for real-time monitoring because network events continuously enter the system. The inference engine can immediately apply rules to newly observed events.

However, backward chaining is useful during investigation because security analysts may begin with a specific query such as:

HighPriorityAlert(User101)?

and work backward to determine why the alert should be generated.

Therefore, a practical cybersecurity system can use both techniques together.

---

**16. Final Inference**

The cybersecurity system receives the following observations:

1. User101 has five failed login attempts.
2. User101 logged in from an unusual location.
3. User101 accessed a confidential file without authorization.

Using the knowledge base and inference rules, the system derives:

FiveFailedLogins(User101)
↓
BruteForceAttack(User101)
↓
SuspiciousActivity(User101)
↓
SecurityBreach(User101)
↓
HighPriorityAlert(User101)

Thus, the system concludes that User101's activity represents a possible security breach and a high-priority security alert should be generated.The system can then recommend that the security team investigate User101's account, verify the login activity, review the accessed files, and take appropriate security measures.

**17. Advantages of the Rule-Based Cybersecurity System**

**17.1 Explainability**

The system can explain why an alert was generated by showing the chain of facts and rules.

**17.2 Fast Decision Making**

Logical rules can quickly identify known patterns of suspicious behavior.

### 17.3 Consistency
The same rules are applied consistently to different users and security events.
### 17.4 Easy Rule Modification
Security administrators can add, remove, or modify rules as new attack patterns are discovered.
### 17.5 Useful for Real-Time Monitoring
Forward chaining can process incoming security events and generate alerts quickly.

## 18. Limitations
### 18.1 Dependence on Known Rules
The system may fail to detect completely new attacks that are not represented in the knowledge base.
### 18.2 Large Knowledge Base
As the number of cybersecurity threats increases, the number of rules can become very large.
### 18.3 False Positives
Legitimate activities may sometimes match suspicious activity rules and generate unnecessary alerts.
### 18.4 Rule Maintenance
Cybersecurity rules must be regularly updated because attackers continuously develop new techniques.
### 18.5 Limited Learning Capability
A traditional rule-based system does not automatically learn new attack patterns from data unless machine-learning techniques are integrated.

## 19. Knowledge Engineering and Rule Maintenance
Knowledge engineering helps maintain the cybersecurity knowledge base in a structured way. When a new type of attack is discovered, cybersecurity experts can:

1. Identify the new attack pattern.
2. Convert the pattern into logical predicates.
3. Create a new rule.
4. Add the rule to the knowledge base.
5. Test the rule against existing facts.
6. Validate whether it produces correct conclusions.
7. Deploy the updated knowledge base.

For example, if security experts discover that repeated password-reset requests followed by unusual login activity indicate account takeover, a new rule can be added:

$$\textbf{RepeatedPasswordReset}(x) \land \textbf{UnusualLocation}(x) \rightarrow \textbf{AccountTakeoverRisk}(x)$$

This makes the system easier to maintain and adapt to changing cybersecurity threats.

## 20. Conclusion
The Cybersecurity Threat Detection System demonstrates how knowledge representation and logical inference can be applied to a real-world cybersecurity problem.The system represents cybersecurity knowledge using Propositional Logic and First-Order Logic. A knowledge base containing facts and rules is constructed to represent observed activities and cybersecurity expertise.

Using unification and substitution, general rules can be applied to specific users. Forward chaining starts from observed events and derives the final security alert, while backward chaining starts from a desired query and determines whether the necessary conditions are satisfied. Resolution provides a formal method for proving the security-breach conclusion.

For User101, the system successfully derives:

**Five Failed Logins → Brute-Force Attack → Suspicious Activity → Security Breach → High-Priority Alert**

Therefore, the final inference is:
**HighPriorityAlert(User101) = TRUE**
This demonstrates that a rule-based knowledge system can provide an explainable and systematic approach to cybersecurity threat detection. Combining forward chaining for real-time monitoring with backward chaining for investigation can make the system more effective in practical cybersecurity environments.

## 21. References


1. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Pearson.
2. Elaine Rich, Kevin Knight and Shivashankar B. Nair, *Artificial Intelligence*, McGraw-Hill.
3. George F. Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, Pearson.
4. Knowledge Representation and Reasoning concepts – Propositional Logic, First-Order Logic, Unification, Resolution and Inference.
5. Cybersecurity threat detection and rule-based expert systems – conceptual application.